Available online at www.sciencedirect.com

ScienceDirect

Transportation Research Procedia 91 (2025) 433–440

Transportation Research Procedia

www.elsevier.com/locate/procedia

The Science and Development of Transport - TRANSCODE 2025

Predicting public transport delay times from Real-Time GTFS data

Kristijan Jurić[a], Tomislav Erdelić[a,*], Nikola Mardešić[a], Tonči Carić[a]

[a] *University of Zagreb, Faculty of Transport and Traffic Sciences, Vukelićeva 4, Zagreb 10000, Croatia*

Abstract

In the rapidly evolving field of intelligent transport systems, real-time data plays a crucial role in improving the efficiency and reliability of public transportation. This paper presents a predictive modeling to forecast public transport delays from Real-Time General Transit Feed Specification (GTFS-RT) data provided by Zagrebački Električni Tramvaj. To manage real-time predictions, a web application has been developed that processes, displays, and stores both static and real-time data. Additionally, it enables users to track the current locations of vehicles and schedules, as well as delay predictions per stops. The results indicate a mean absolute delay percentage prediction error of up to 20% for routes with sparse data, and around 10% for routes with dense data. Findings underscore the potential of integrating intelligent transport systems with predictive analytics to enhance public transit services and user experiences in rapidly growing urban areas, contributing to more efficient urban mobility solutions.

© 2025 The Authors. Published by ELSEVIER B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the Science and Development of Transport - TRANSCODE 2025

Keywords: GTFS; GTFS-RT; public transport; prophet; ITS

1. Introduction

In modern cities such as Zagreb, Croatia, public transportation systems play a vital role in urban mobility. As transportation infrastructure continues to expand, the demand for efficient and reliable solutions increases alongside urban growth. This highlights the necessity for real-time, data-driven approaches to managing public transport operations, (Bruglieri et al. (2015); Amorim and Silva (2025)), especially within the domain of Intelligent Transport Systems (ITS), (Nuzzolo and Comi (2016); Wang and Loo (2024)). When dealing with public transport real-time data, data standards like the General Transit Feed Specification (GTFS) (McHugh (2013)), Network Timetable Exchange (NeTEx) (Bourée et al. (2015)), and Standard Interface for Real-time Information (SIRI) (Reed (2013)), among others, are crucial for facilitating data exchange. As this paper focuses on GTFS Real-Time (GTFS-RT) data, we provide a review of recent research on GTFS usage.

Liu et al. (2024) used GTFS-RT to measure the impacts of disruptions on public transit accessibility and reliability. A study by Wu et al. (2020) introduced a convolutional long short-term memory model with a self-attention mechanism for predicting bus travel times using GTFS data. Kumar et al. (2018) proposed a method for inferring the most likely trajectory of individual passengers based on automatic fare collection data and GTFS. Additionally, Alsger

* Corresponding author. Tel.: +385 1 245 7951
E-mail address: terdelic@fpz.unizg.hr

2352-1465 © 2025 The Authors. Published by ELSEVIER B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the Science and Development of Transport - TRANSCODE 2025
10.1016/j.trpro.2025.10.056

et al. (2018) integrated household travel surveys, origin-destination surveys, and GTFS data to identify passengers' trip purposes. Du (2019) presented a framework for analyzing and estimating the travel purposes of transit passengers, offering an alternative to traditional survey methods. Prommaharaj et al. (2020) and Fortin et al. (2016) showcased the utility of GTFS data, specifically through a visualization tool that highlights spatial and temporal patterns in transit services. Wessel et al. (2017) introduced a method to improve the accuracy of GTFS data by utilizing real-time vehicle location datasets. In a later study, Wessel and Farber (2019) evaluated the spatial and temporal accuracy of GTFS schedules, finding that areas with lower scheduled access tend to have less reliable estimates and greater variability in accessibility. Chondrodima et al. (2022) applied particle swarm optimization and neural networks to predict public transportation arrival times using GTFS data. Vieira et al. (2023) proposed a method for estimating emissions using GTFS data. Compared to previous research, this paper aims to further enhance the application framework for handling GTFS data and apply a machine learning method to predict vehicle delays.

The objective of the paper is to develop a web framework for handling GTFS-RT and a machine learning model to predict delay times at each stop in the transport network. The rest of the paper is organized as follows. Section 2 describes the GTFS data standard. Section 3 describes the architecture and functionality of the developed system for retrieving, processing and visualizing the real-time GTFS data. Section 4 describes the machine learning model used to predict vehicle delay times. Section 5 presents the results of the developed application and implement model. The conclusion of the paper is given in Section 6.

2. Data

2.1. GTFS standard

GTFS is a standardized data format that transit agencies utilize to make their transit information publicly available. This data, generated during the operation of buses, trams, trains, and ferries, is shared with both transport agencies and the general public (Google Developers (2024)). Transit providers distribute key details such as schedules, stop locations, departure and arrival times, as well as fare information through the GTFS format, making it readily accessible for users relying on mobile applications or web platforms for trip planning and monitoring (Google Developers (2024)). GTFS data is divided into two primary components that are interrelated and complement one another from a systematic viewpoint. One of the key components is real-time data, which delivers a constant stream of information regarding the status of transit services. This includes updates on vehicle locations, delays, service interruptions, and arrival and departure times, all of which are relayed to transit applications, keeping users informed about the current availability and conditions of transit services. The main entities in GTFS Real-Time (GTFS-RT) data include *trip updates*, which represent modifications to the scheduled transit timetable, providing arrival and departure times, though these details may sometimes be inaccurate or missing. *Trip updates* are essential for addressing dynamic transit operations, such as delays, additional trips, or cancellations. *Vehicle positions* deliver real-time coordinates of a vehicle's current location, helping track its movement within the transit network. *Service alerts* notify users about disruptions affecting the transit network, such as line suspensions, stop closures, or larger network-wide issues caused by operational problems. *Trip modifications* provide information about changes to active or planned routes, which may include new route shapes or temporary stops on detours. GTFS static data consist of a collection of text files detailing the operation of a transit system. Unlike real-time data, which is updated frequently, static data offers long-term scheduling information. The are several files that provide crucial information. *Agency.txt* contains essential information about the transit agency, such as its name, website, and contact details. *Stops.txt* lists all stops in the transit system for the schedule period, defining where passengers can board or leave transit vehicles with geographic coordinates. *Routes.txt* outlines all routes within the transit network, including each route's unique identifier and mode of transportation. The *trips.txt* file links individual trips to their respective routes, showing their schedule, path, and direction. The *stop_times.txt* file provides the specific time a transit vehicle is expected to arrive or depart from each stop. The *calendar.txt* file specifies which days services are running, while *fare_attributes.txt* includes details about fare pricing and payment methods.

2.2. Data from Zagrebački Električni Tramvaj (ZET)

For the case study in this paper, we selected data from Zagreb's public transportation system, operated by the company ZET. This area and dataset were chosen to leverage both the authors' knowledge and experience as users

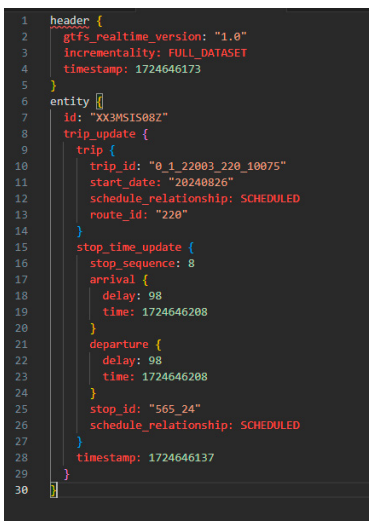
of Zagreb's public transportation, as well as to address the raised research questions. Zagreb's public transportation system comprises various modes of transit, including trams, buses, railways, and cable cars, which collectively provide broad coverage throughout the city. In this paper we focus primarily on the tram and bus networks, as they are represented within the GTFS framework in terms of data availability. The tram network spans 148 kilometers, it is served by 15 daytime and 4 nighttime lines, with 256 tram stops strategically placed to minimize walking distances, particularly in areas near hospitals, schools, and public institutions. Additionally, the bus network, which includes 148 daytime and 4 nighttime lines, plays a crucial role in connecting the city's outer areas to the central parts, as illustrated in Fig. 1a. The network features 2,103 bus stops. On weekdays during peak hours, approximately 303 buses are in operation, while 185 buses run on Saturdays and 123 on Sundays and public holidays. To collect ZET GTFS data, the application described in this paper uses Python in combination with MySQL database. The real-time data feed is accessible from the ZET server at the link https://zet.hr/gtfs-rt-protobuf. Typically, the refresh rate of the data within the protocol buffer ranges between 10 and 15 seconds. Data is obtained using libraries such as *"gtfs_realtime_pb2"* for managing GTFS real-time feeds, *"protobuf_to_dict"* for converting protocol buffer messages to Python dictionaries, and *"requests"*, a Python library that handles HyperText Transfer Protocol (HTTP) requests to access trip updates. This Python dictionary data can then be fully integrated with data manipulation libraries such as *"pandas"* for improved handling and analysis. The data extracted from the ZET GTFS feed protocol buffer is illustrated in Fig. 1b, which shows that ZET GTFS adheres to the standard specified by Google, including most of the key data fields related to trip update in real-time feeds. However, it is important to note, that exact vehicle positions are not provided in the ZET GTFS feed. Once the real-time data is extracted, it is merged with the static information to create a unified dataset. The data is stored in a MySQL database, which organizes both real-time feeds and static data into tables with defined rows and columns, presented in Fig. 2. Each table corresponds to a specific entity, such as routes, stops, or trips. Relationships between tables are established through the use of primary and foreign keys, forming a relational GTFS database. In addition to the static data, real-time data was collected and stored in the *"zet_feed_data"* table. This data was gathered by the application from July 24, 2023, to August 16, 2023, resulting in a total of 35,958,139 records, occupying 5.73 GB of storage.

3. Application framework

The application presented in this paper is designed using a frontend and back-end architecture, with the connection between them handled by the internal mechanisms of the framework in use. The primary framework utilized is Dash, a Python-based platform that creates a seamless integration between backend data processing and frontend visualization. The diagrams of backend and frontend systems are presented in Figs. 3a and 3b. The system is organized into several components or classes. The *GtfsDataDownloader* is responsible for acquiring and downloading static GTFS



(a) ZET network



```
header {
  gtfs_realtime_version: "1.0"
  incrementality: FULL_DATASET
  timestamp: 1724646173
}
entity {
  id: "XX3MSIS08Z"
  trip_update {
    trip {
      trip_id: "0_1_22003_220_10075"
      start_date: "20240826"
      schedule_relationship: SCHEDULED
      route_id: "220"
    }
    stop_time_update {
      stop_sequence: 8
      arrival {
        delay: 98
        time: 1724646208
      }
      departure {
        delay: 98
        time: 1724646208
      }
      stop_id: "565_24"
      schedule_relationship: SCHEDULED
    }
    timestamp: 1724646137
  }
}
```

(b) Trip updates feed

Fig. 1. ZET network and real-time data response

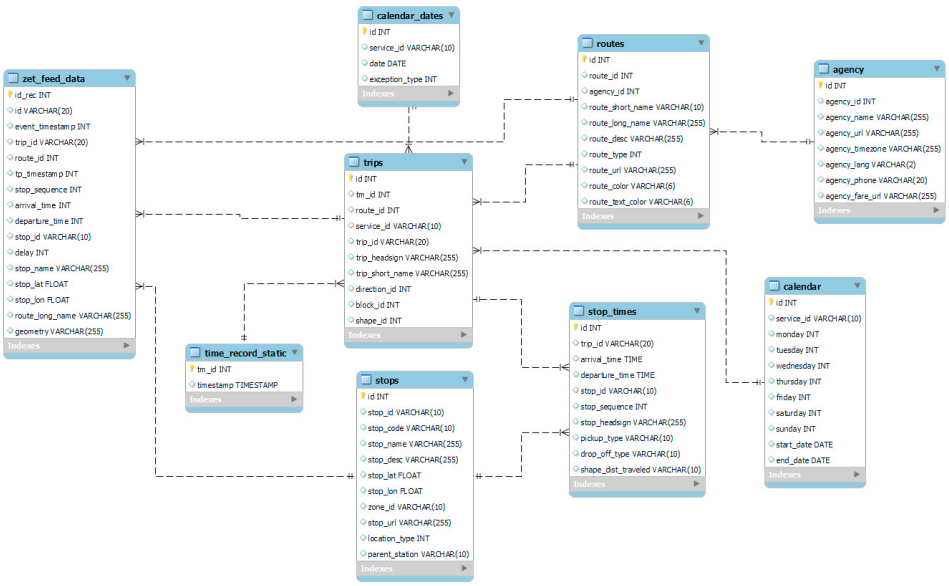


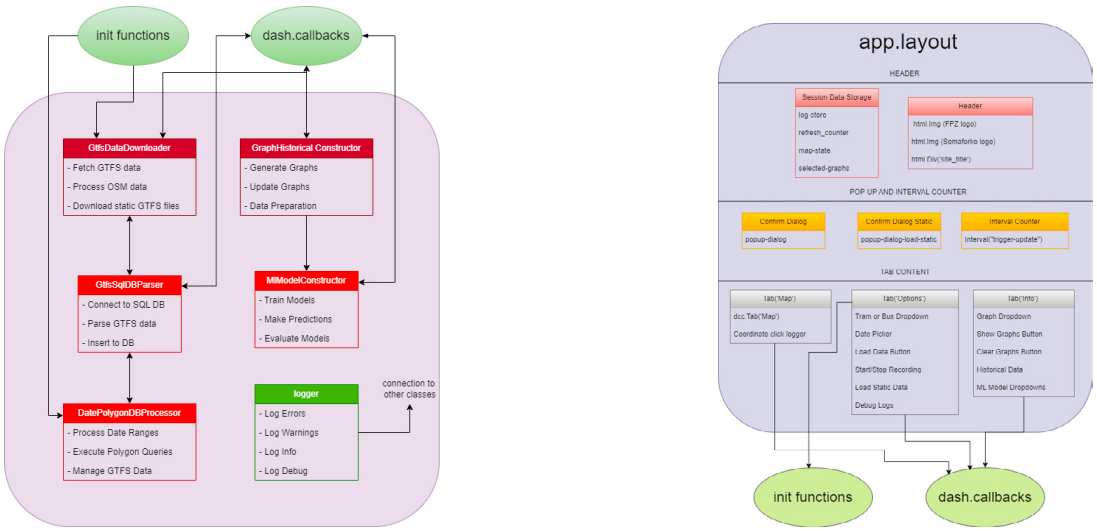
Fig. 2. MySQL database diagram

data, as well as initializing and obtaining OpenStreetMap (OSM) data for Zagreb, which is used to extract polylines for route trajectory visualization. The *GtfsSqlDBParser* manages and stores GTFS data within a MySQL database, handling both static and real-time data. The *GTFS Public Transport Net Constructor* filters bus and tram data extracted from OSM based on specific criteria such as route IDs and stop IDs, enabling detailed analysis of routes and stops. The *DatePolygonDBProcessor* processes database records within specified date ranges and polygon coordinates, supporting spatial and temporal queries to extract relevant data for further analysis. The *GraphHistoricalConstructor* creates various graphs for visualizing historical delay data, enabling users to generate graphs such as average delay per stop, delay distribution histograms, on-time performance pie charts, and others, and also supports machine learning error metrics for model comparison. The *Backup and Scheduling System* handles scheduled tasks for long-term data recording and includes an automated backup system for the MySQL database to ensure data availability in case of critical errors during data insertion or retrieval. Lastly, the *LogParser* manages logging events within the application server, capturing log entries with different message levels (e.g., info, warning, error).

Frontend development focuses on the user's interaction with the application through the web browser. The application's frontend structure is based on a single-page layout, as depicted in Fig. 3b, which outlines the frontend architecture. The component *layout* can be categorized as the frontend layer of the Dash application, which can be interacted through Python code or directly through Hypertext Markup Language (HTML), Cascading Style Sheets (CSS) and JavaScript. This contains the structure of the user interface elements, that can be dropdowns, buttons, graphs and etc. The element "callback" interacts between the user interface and the backend functions by wrapping them as a decorator. At the top, the "header" component contains the logos and application titles. Below it, there is a section that handles interactions requiring confirmation from the user. Interval counters are implemented to provide periodic interface updates, which are crucial for regularly refreshing vehicle positions on the map (vehicle position is updated when vehicle arrives at the stop). The tab content is divided into three sections: the *Map* tab, which displays real-time data on the map, including current vehicle positions for selected routes; the *Options* tab, allowing users to choose specific routes or batches via "Tram" or "Bus Dropdowns" to be visualized in the "Map" tab, as well as providing controls to start or stop GTFS data recording and load static data; and the *Info* tab, used for showing graphs and the results of machine-learning models.

4. Trend prediction modeling

In this paper we implement machine learning model well suited for time-series data to predict transport delays, a Meta's tool called Prophet (Taylor and Letham (2017)). Prophet is best accustomed to time-series forecasting with large quantities of data, which can predict quite well even if missing data exists. The primary variable used for predictions is the delay variable. This variable is important because, if predicted, it can increase the efficiency of the overall public transportation system evaluated. The model focuses on scenarios with strong seasonal effects and non-linear



(a) Backend (b) Frontend

Fig. 3. Diagrams of backend and frontend

trends, like those present with delay prediction when, for example, there is no data between 11:00 PM and 4:20 AM. The time-series data obtained gets segmented into multiple linear segments. The equations 1-3 represent the general form of Prophet model (Taylor and Letham (2017)). The trend component, denoted as $g(t)$, models non-periodic variations in the time series, where k represents the base growth rate, $a(t)$ is an indicator vector marking whether time t has crossed a changepoint, δ captures the rate adjustments at each changepoint, and m and γ are parameters that correct the offset at these changepoints. The seasonal component, $s(t)$, accounts for regular fluctuations, such as weekly or yearly patterns, where P defines the length of the seasonal cycle (e.g., $P = 365.25$ for annual patterns), N is the number of Fourier terms, and a_n and b_n are the coefficients estimated from the data. The holiday effect, $h(t)$, captures the impact of holidays or special events, where κ_i quantifies the effect of holiday i on the time series, and $\mathbf{1}_{t\in D_i}$ is an indicator function (dummy variable) that equals 1 if t is within the holiday set D_i, and 0 otherwise. This indicator function is a simple way to encode binary information. Finally, $e(t)$ represents the error term, which is assumed to follow a normal distribution and accounts for any unique noise in the data. Prophet models are commonly fitted using maximum a posteriori estimation. The fitting process involves calibrating key parameters, such as k, m, δ, as well as coefficients related to seasonality and holiday effects, to accurately capture the observed patterns in the data (Taylor and Letham (2017)). For the ZET GTFS data, ensuring data cleanliness is essential to avoid unnecessary outliers that

$$y(t) = g(t) + s(t) + h(t) + e(t) \quad (1) \qquad g(t) = (k + a(t)^T\delta)t + (m + a(t)^T\gamma) \quad (2)$$

$$s(t) = \sum_{n=1}^{N}\left(a_n\cos\left(\frac{2\pi nt}{P}\right) + b_n\sin\left(\frac{2\pi nt}{P}\right)\right) \quad (3) \qquad h(t) = \sum_i \kappa_i \mathbf{1}_{t\in D_i} \quad (4)$$

could negatively impact the model's accuracy. Some outliers can be extreme (e.g., sudden delays of -1,498), which do not reflect normal conditions. These outliers can distort the trend and seasonality patterns in Prophet, leading to less accurate predictions, and are therefore removed using the Z-score and Interquartile Range (IQR) methods. The Z-score method detects outliers by measuring their deviation from the mean. Each data point is subtracted from the mean and then divided by the standard deviation. If a point's Z-score is significantly far from zero, it is classified as an outlier and removed. The IQR method identifies outliers by examining the data's spread between the 25th percentile (Q1) and the 75th percentile (Q3). If a data point falls below $Q1 - 1.5 \cdot IQR$ or above $Q3 + 1.5 \cdot IQR$, it is considered an outlier. In addition to outlier handling, missing *NaN* values were addressed by dropping them to ensure data quality. Notably, columns like *"arrival_time"* had 56.85% missing values indicating that over half of the data was unusable and needed to be discarded to maintain accuracy in predictions.

The temporal granularity of 10-15 seconds for GTFS-RT updates provides high-resolution data, which can improve Prophet's prediction accuracy by capturing short-term variability, but smoothing the data through aggregation (e.g., to 1-minute intervals) might reduce noise and enhance model performance, particularly when analyzing broader patterns or mitigating overfitting to fluctuations.

5. Results

5.1. Web application

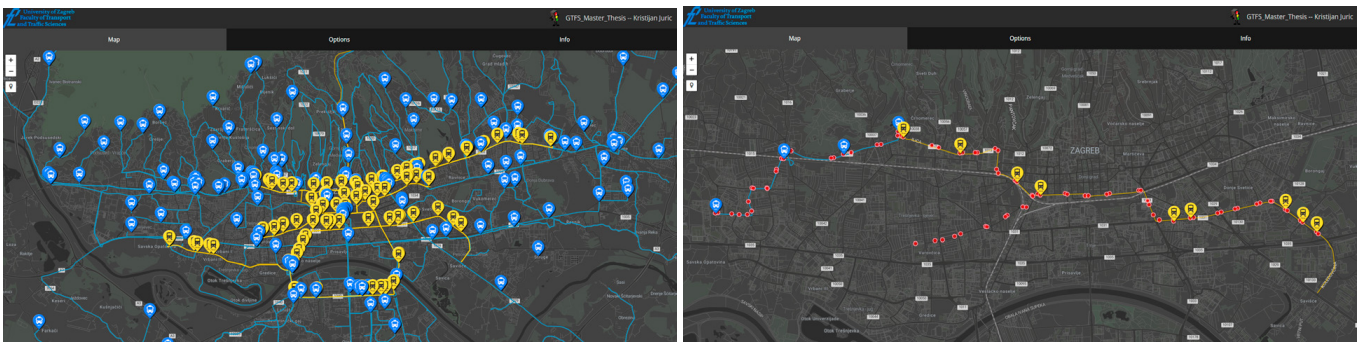
The complete Python code for real-time data processing, visualization and delay prediction is available at the following link https://dev.azure.com/cronical-studios/_git/GTFS_master_thesis. The loaded web page application used for collection and storing is presented in Fig. 4a. Specific locations of trams and buses can be seen, which are represented by yellow pins for trams and blue pins for buses. If in the *Options* tab, a specific routes are selected, then only them are visualized as in Fig. 4b, where red points represent routes' stops. To keep the paper concise, the other functionalities of the developed application, such as data storage, spatial queries, historical data visualization, etc., are available in video link https://youtu.be/uq8LeRXSDeI?si=i8lMb56IdtqdIJQI.

5.2. Delay prediction

For the evaluation purpose, one tram route (7) and one bus route (136) were used, presented on Fig. 5. Route 7, illustrated in Fig. 5a, travels between *Savski Most* and *Dubec* in Zagreb. A shown in Fig. 5c the stops with most data are *Tržnica Kvatrić*, *Mašićeva*, and *Trg P. Krešimira*. These stops have a significant amount of data recorded, with 2,000 to 3,000 data points each. There are several stops with low data collection, indicating that some parts of the route were not consistently recorded. Route 136 is a bus route that connects *Črnomerec* and *Špansko*, as seen in Fig. 5b. The data points on Fig. 5d show that stops such as *Vrapčanska* and *I. Brlić Mažuranić* have the highest number of data points collected, with over 500 records each. This indicates a high frequency of recordings at these stops compared to other bus stops.

The ratio of training to testing data was 80/20, indicating that 80% of the dataset was utilized for calibrating the model, whereas the remaining 20% was set aside for validating the model's performance. The Prophet object is initialized with weekly and daily seasonality. To evaluate the model the following well-known metrics in the research field were used: Root Mean Squared Error (RMSE, $\sqrt{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$, where y_i is the actual value and $\hat{y}_i$ is the predicted value), Mean Absolute Error (MAE, $\sum_{i=1}^{n}|y_i - \hat{y}_i|$), Mean Absolute Percentage Error (MAPE, $\frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$), Symmetric Mean Absolute Percentage Error (SMAPE, $\frac{100\%}{n}\sum_{i=1}^{n}\frac{|y_i - \hat{y}_i|}{(|y_i| + |\hat{y}_i|)/2}$).

The model's evaluation results for four stops on each of the two observed routes are presented in Table 1. The prediction performance for *Route 7* remains relatively stable, with *Tržnica Kvatrić* and *Trg P. Krešimira* showing MAPE values close to 10%, indicating that the model accurately predicts delays at these stops. Compared to some other investigated stops, this stop has relatively low error, likely due to the consistent traffic patterns and stop utilization along these sections. Conversely, *Mašićeva* and *Držićeva* exhibit higher error rates, with *Mašićeva* reaching a MAPE of 15.87%. These stops might be more susceptible to unpredictable delays, potentially due to their location near busy areas or frequent interruptions along the route, leading to increased variability in arrival times. In contrast, the model performs better on *Route 136*, particularly at stops like *Vrapčanska*, where the values for RMSE and MAE are notably low (11.84s and 10.04s, respectively). The MAPE also remains at a moderate level, suggesting a satisfactory level



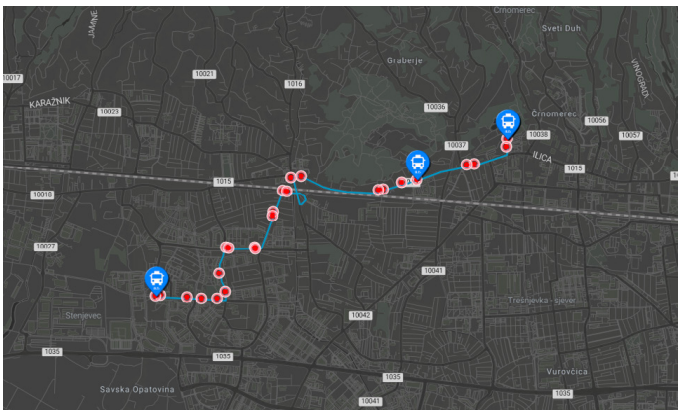
(a) Map with all tram and bus data

(b) Map with selected route

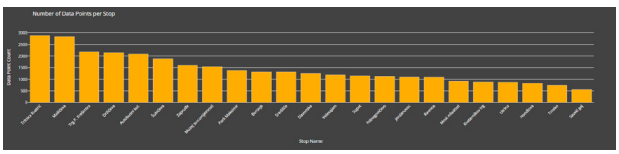
Fig. 4. Web application



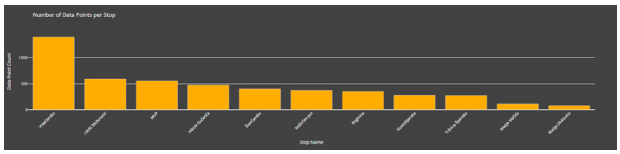
(a) Tram route 7



(b) Bus route 136



(c) Data points route 7



(d) Data points route 136

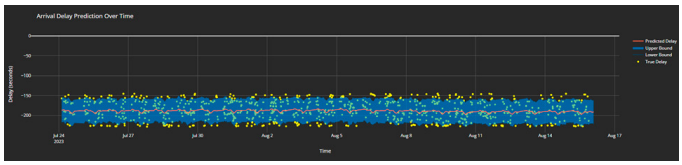
Fig. 5. Selected routes

Table 1: Performance metrics for routes 7 and 136

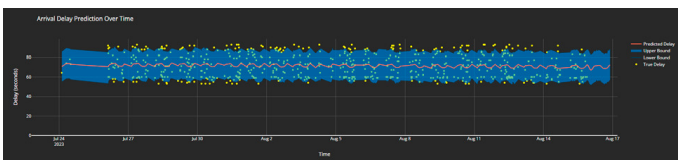
Route 7				
Stop	**RMSE (s)**	**MAE (s)**	**MAPE (%)**	**SMAPE (%)**
Tržnica Kvatrić	23.35	19.75	10.76	10.54
Mašićeva	30.84	26.65	15.87	15.48
Trg. P.Krešimira	23.58	20.07	10.87	10.73
Držićeva	35.58	30.38	15.32	14.93
Route 136				
Stop	**RMSE (s)**	**MAE (s)**	**MAPE (%)**	**SMAPE (%)**
Vrapčanska	11.84	10.04	14.86	14.56
Ivane Brlić Mažuranić	15.57	13.52	14.66	14.88
MUP	7.21	6.21	5.36	5.33
Nikole Gučetića	8.39	7.4	19.02	18.02

of prediction accuracy. However, stop such as *Nikole Gučetića* displays a high MAPE (19.02%), implying that the model's predictions for these locations are less reliable. This may result from unpredictable variables, such as road conditions or fluctuations in bus operations, which create difficulties for the model to manage effectively.

Concerning the prediction of arrival delays using the Prophet model for specific stops on each of the observed route, the predictive accuracy is illustrated in Fig. 6. These visual representations compare the actual delays (shown as yellow points) with the predicted delays (represented by the red line), offering insights into the model's precision for each route. For *Savski Most - Dubrava*, Fig. 6a shows that the model effectively captures the general delay trend, despite the presence of a few outliers. The red line of predicted delays aligns well with the yellow points of recorded delays, reflecting a satisfactory degree of accuracy. Moreover, the blue-shaded area representing the prediction interval remains stable, indicating that the Prophet model is provided with reliable data for producing confident forecasts at this stop. Similarly, Fig. 6b demonstrates the model's accurate prediction for *Route 136* at *Črnomerec - Španska*, with fewer major outliers. The narrower prediction intervals here suggest a higher confidence in this particular forecast.



(a) Route 7 – Savski Most - Dubrava



(b) Route 136 – Črnomerec - Španska

Fig. 6. Prediction graphs

The consistently positive recorded delays imply that vehicles on this route frequently arrive later than scheduled, a pattern that the model has successfully captured.

6. Conclusion

Within this paper we developed an application to visualize real-time transit data for Zagreb and predict vehicle delay times using machine learning. The application processed GTFS data for key tram and bus routes, focusing on vehicle positions and delays. In delay prediction, the Prophet model was used and evaluated using metrics like RMSE, MAE, MAPE, and SMAPE. Predictions were more accurate for routes with dense data, but less so for routes with inconsistent data. This highlighted the need for higher-quality, continuous data to improve forecasting accuracy. For future research, it would be valuable to explore the integration of additional data sources, such as weather and traffic conditions, to further enhance delay prediction accuracy. The limitations of the current model include its reliance on the quality and consistency of GTFS-RT data, which may affect predictions for routes with sparse data. Additionally, the generalizability of the model could be improved by testing it on various cities with different transportation networks, data standards and data availability.

References

Alsger, A., Tavassoli, A., Mesbah, M., Ferreira, L., Hickman, M., 2018. Public transport trip purpose inference using smart card fare data. Transportation Research Part C: Emerging Technologies 87, 123–137. doi:https://doi.org/10.1016/j.trc.2017.12.016.

Amorim, J., Silva, J., 2025. The influence of residential location and public transit options on commuters' intention to use travel apps for different travel purposes. Travel Behaviour and Society 38, 100913. doi:https://doi.org/10.1016/j.tbs.2024.100913.

Bourée, K., Arneodo, F., Tibaut, A., 2015. Transmodel-netex-eptis: a european standard for modelling and exchanging fares. v, in: ITS World Congress 2015: proceedings. 22nd World Congress and Exhibition on Intelligent Transport Systems and Services, Bordeaux, France.

Bruglieri, M., Bruschi, F., Colorni, A., Luè, A., Nocerino, R., Rana, V., 2015. A real-time information system for public transport in case of delays and service disruptions. Transportation Research Procedia 10, 493–502. doi:https://doi.org/10.1016/j.trpro.2015.09.003. 18th Euro Working Group on Transportation, EWGT 2015, 14-16 July 2015, Delft, The Netherlands.

Chondrodima, E., Georgiou, H., Pelekis, N., Theodoridis, Y., 2022. Particle swarm optimization and rbf neural networks for public transport arrival time prediction using GTFS data. International Journal of Information Management Data Insights 2, 100086. doi:https://doi.org/10.1016/j.jjimei.2022.100086.

Du, B., 2019. Estimating travellers' trip purposes using public transport data and land use information .

Fortin, P., Morency, C., Trépanier, M., 2016. Innovative GTFS data application for transit network analysis using a graph-oriented method. Journal of Public Transportation 19, 18–37. doi:https://doi.org/10.5038/2375-0901.19.4.2.

Google Developers, 2024. General transit feed specification (GTFS) reference. https://developers.google.com/transit/gtfs/reference. Accessed: 2024-08-24.

Kumar, P., Khani, A., He, Q., 2018. A robust method for estimating transit passenger trajectories using automated data. Transportation Research Part C: Emerging Technologies 95, 731–747. doi:https://doi.org/10.1016/j.trc.2018.08.006.

Liu, L., Porr, A., Miller, H.J., 2024. Measuring the impacts of disruptions on public transit accessibility and reliability. Journal of Transport Geography 114, 103769. doi:https://doi.org/10.1016/j.jtrangeo.2023.103769.

McHugh, B., 2013. Pioneering open data standards: The GTFS story. https://beyondtransparency.org/part-2/pioneering-open-data-standards-the-gtfs-story/. Accessed: 2024-08-24.

Nuzzolo, A., Comi, A., 2016. Advanced public transport and intelligent transport systems: new modelling challenges. Transportmetrica A: Transport Science 12, 674–699. doi:10.1080/23249935.2016.1166158, arXiv:https://doi.org/10.1080/23249935.2016.1166158.

Prommaharaj, P., Phithakkitnukoon, S., Demissie, M.G., Kattan, L., Ratti, C., 2020. Visualizing public transit system operation with GTFS data: A case study of calgary, canada. Heliyon 6.

Reed, L.T., 2013. Real-time transit passenger information: A case study in standards development. Phd thesis. Georgia Institute of Technology. Available at https://repository.gatech.edu/entities/publication/11e3ee44-d72a-49d0-af41-019c15b69300.

Taylor, S.J., Letham, B., 2017. Forecasting at scale. PeerJ Preprints 5, e3190v2. doi:10.7287/peerj.preprints.3190v2. cC BY 4.0 Open Access.

Vieira, J.P.B., Pereira, R.H., Andrade, P.R., 2023. Estimating public transport emissions from general transit feed specification data. Transportation Research Part D: Transport and Environment 119, 103757. doi:https://doi.org/10.1016/j.trd.2023.103757.

Wang, H., Loo, B.P., 2024. The public transport disadvantaged in a highly transit-oriented city: An analytical framework, key challenges and opportunities. Journal of Transport Geography 120, 103983. doi:https://doi.org/10.1016/j.jtrangeo.2024.103983.

Wessel, N., Allen, J., Farber, S., 2017. Constructing a routable retrospective transit timetable from a real-time vehicle location feed and GTFS. Journal of Transport Geography 62, 92–97. doi:https://doi.org/10.1016/j.jtrangeo.2017.04.012.

Wessel, N., Farber, S., 2019. On the accuracy of schedule-based GTFS for measuring accessibility. Journal of Transport and Land Use 12, 475–500. URL: https://www.jstor.org/stable/26911278.

Wu, J., Wu, Q., Shen, J., Cai, C., 2020. Towards attention-based convolutional long short-term memory for travel time prediction of bus journeys. Sensors 20. doi:10.3390/s20123354.